

Zero-Knowledge Proofs (ZKP): Advanced Cryptography and E-KYC Architecture

Practice School-I (PS-1) Technical Report
Department of Computer Science and Information Systems
BITS Pilani, K. K. Birla Goa Campus

29 May, 2026

Contents

1	Theoretical Foundations of ZKP	2
1.1	The Layman's Analogy: What is a ZKP?	2
1.2	The Three Pillars of ZKP	2
1.3	Mathematical Prerequisites	2
1.4	ZK-Protocols: SNARKs vs. STARKs	3
2	Core Entities & The Operational Mechanism	3
2.1	The Three Core Entities in a ZKP	3
2.2	The Mechanism: How a ZKP Operates	3
2.3	The Trusted Setup (CRS) & Multi-Party Computation	4
3	Under the Hood: The Compilation Pipeline	5
4	Implementation 1: The "Sum == 12" Circuit	5
4.1	In Plain English: What is actually happening?	5
4.2	Data Flow Diagram	6
4.3	The Privacy Layer: Blinding Factors	6
4.4	Circom Source Code	6
4.5	Compilation and Execution: The Lifecycle of the Proof	7
5	Implementation 2: E-KYC Age Verification (≥ 18)	7
5.1	In Plain English: What is actually happening?	7
5.2	Verification Architecture	8
5.3	The Privacy Layer: Merkle Trees & ZK-Friendly Hashing	8
5.4	Circom Source Code	8
5.5	Compilation and Execution: The Lifecycle of the Proof	9

1 Theoretical Foundations of ZKP

1.1 The Layman's Analogy: What is a ZKP?

The Bouncer and the Magic App

Imagine trying to get into a 18+ venue. Normally, you hand your ID card to the bouncer. The bouncer now knows your exact birth date, your full name, your address, and your ID number. You gave up **100%** of your privacy just to prove a **Yes/No** question (Are you over 18?).

A Zero-Knowledge Proof changes this. Instead of an ID card, you have a magic, tamper-proof app on your phone. The app looks at your government ID locally, checks today's date, and verifies you are over 18. The app then spits out a single green checkmark (the proof). You show the bouncer the checkmark. Because the bouncer trusts the cryptography behind the app, they let you in. The bouncer learns absolutely nothing about you except the fact that you are old enough.

A Zero-Knowledge Proof (ZKP) is a cryptographic protocol allowing a **Prover** to convince a **Verifier** of a statement's absolute truth without revealing the data that makes it true. It solves the critical bottleneck in quantitative finance and Web3: verifying regulatory compliance without transferring toxic data liabilities.

1.2 The Three Pillars of ZKP

To be mathematically valid, a ZKP must satisfy three strictly defined properties:

- **Completeness:** If the statement is true, the mathematical proof will perfectly and deterministically convince the verifier.
- **Soundness:** If the statement is false, a malicious prover cannot fake a proof. The probability of forgery is mathematically negligible.
- **Zero-Knowledge:** The proof is merely a randomized string of bytes. It leaks exactly 0% of the underlying data, acting as a one-way mathematical trapdoor.

1.3 Mathematical Prerequisites

Modern zk-SNARKs rely heavily on two cryptographic primitives:

- **Elliptic Curve Cryptography (ECC):** Operations are performed over points on a finite elliptic curve rather than standard integers. This allows for massive cryptographic security with very small key sizes.
- **Bilinear Pairings:** A mathematical mapping that allows the Verifier to check multiplication operations on encrypted data without decrypting it, which is essential for checking the polynomial constraints in a SNARK.

Feature	zk-SNARK (Used in this project)	zk-STARK
Proof Size	Extremely small (~ 1.5 KB), making it ideal for blockchain gas limits.	Large (~ 40 -100 KB), making it expensive to verify on-chain.
Verification Time	Constant time ($O(1)$) regardless of computation size.	Logarithmic time ($O(\log n)$).
Trusted Setup	Required (must generate a CRS).	Not required (completely trustless).
Quantum Resistance	Vulnerable to Shor's algorithm.	Post-Quantum Secure.

1.4 ZK-Protocols: SNARKs vs. STARKs

2 Core Entities & The Operational Mechanism

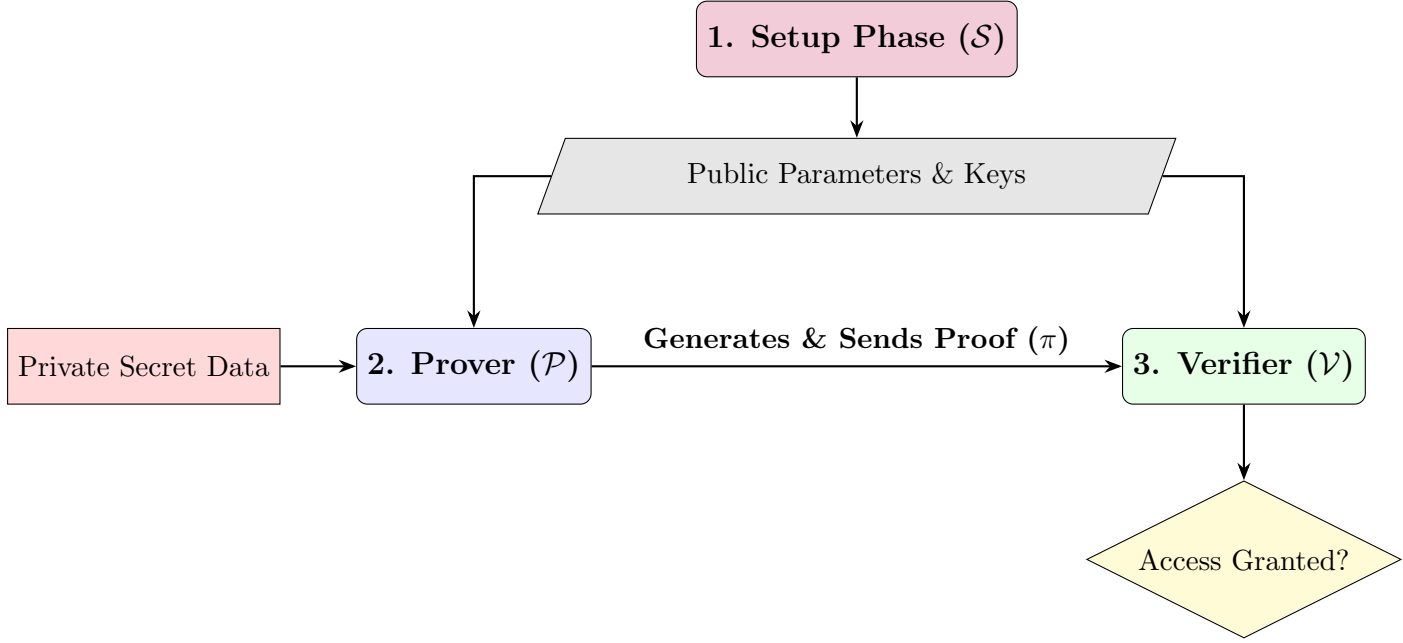
2.1 The Three Core Entities in a ZKP

Every Zero-Knowledge architecture relies on the strict separation and interaction of three fundamental entities:

1. **The Prover ($\mathcal{P}$):** The entity that possesses the hidden knowledge (the secret "witness"). The Prover's goal is to mathematically convince the Verifier that they know the secret without ever revealing the secret itself. (e.g., The user's smartphone holding the DigiLocker ID locally).
2. **The Verifier ($\mathcal{V}$):** The entity requesting the proof. The Verifier does not have the secret and requires absolute mathematical certainty before granting access or approving a transaction. (e.g., The financial institution's Ethereum Smart Contract).
3. **The Setup / Public Parameters ($\mathcal{S}$):** The overarching cryptographic rules, keys, and algorithms that govern the interaction. Both the Prover and Verifier must agree on these exact parameters before any communication begins.

2.2 The Mechanism: How a ZKP Operates

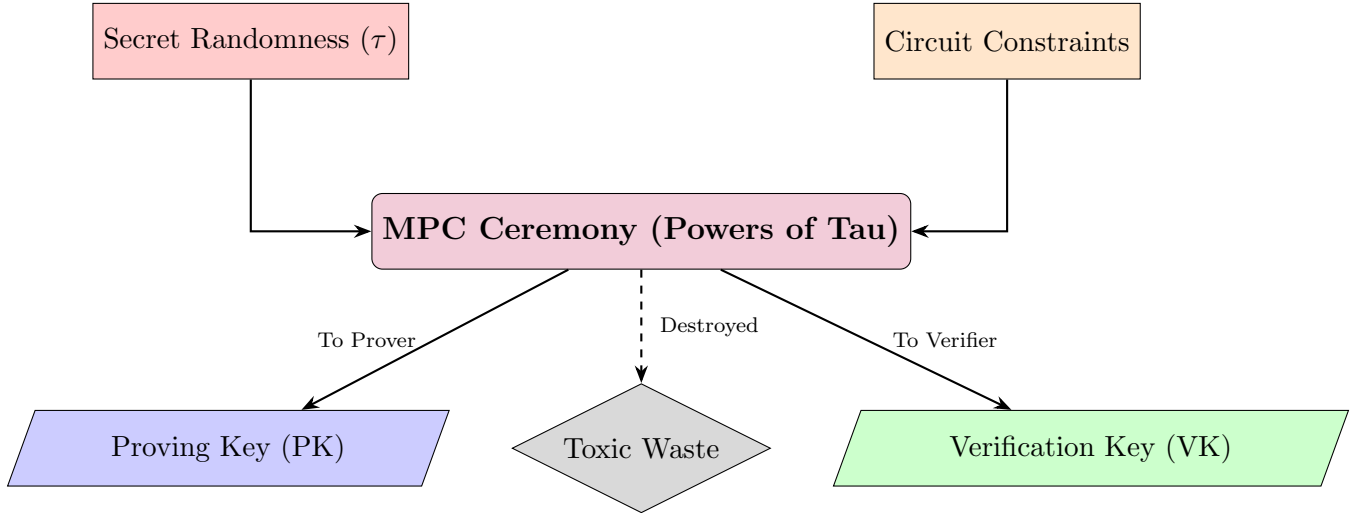
Before diving into the complex mathematics, the operational flow between these three entities works as follows:



2.3 The Trusted Setup (CRS) & Multi-Party Computation

In a *zk-SNARK* system, the **Setup Phase (S)** shown above requires the generation of a **Common Reference String (CRS)**.

To prevent a single entity from having the power to forge proofs, this setup is executed as a **Multi-Party Computation (MPC)** ceremony, often referred to as the *Powers of Tau*.

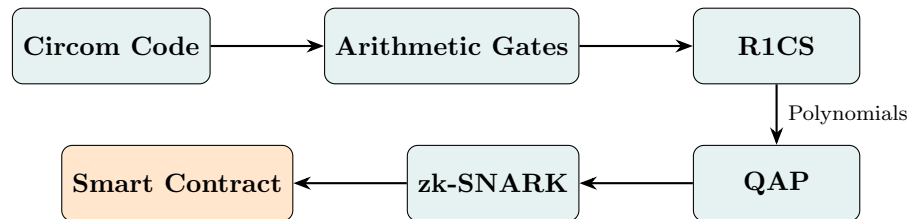


- **Proving Key (PK):** Held by the client device. Used locally to generate the proof.
- **Verification Key (VK):** Held by the Smart Contract. Used publicly to verify the proof.
- **Toxic Waste (τ):** The initial randomness must be permanently destroyed. In an MPC ceremony, hundreds of participants contribute randomness. As long as just

one participant deletes their secret, the entire system is mathematically secure and forgery is impossible.

3 Under the Hood: The Compilation Pipeline

ZKPs do not execute code linearly. A CPU cannot "run" a zero-knowledge proof. Instead, the code is structurally translated into a series of polynomials. This pipeline visualizes how human-readable Circom code becomes a cryptographic SNARK.



- **Arithmetic Gates:** The code is flattened. Complex logic (like loops or 'if' statements) is unrolled into simple addition (+) and multiplication ($\times$) gates.
- **R1CS (Rank-1 Constraint System):** The arithmetic gates are converted into a matrix equation vector format.
- **QAP (Quadratic Arithmetic Program):** The matrices are interpolated into polynomials. By checking a random point on the polynomial, the verifier mathematically checks every single gate in the circuit simultaneously.

4 Implementation 1: The "Sum == 12" Circuit

4.1 In Plain English: What is actually happening?

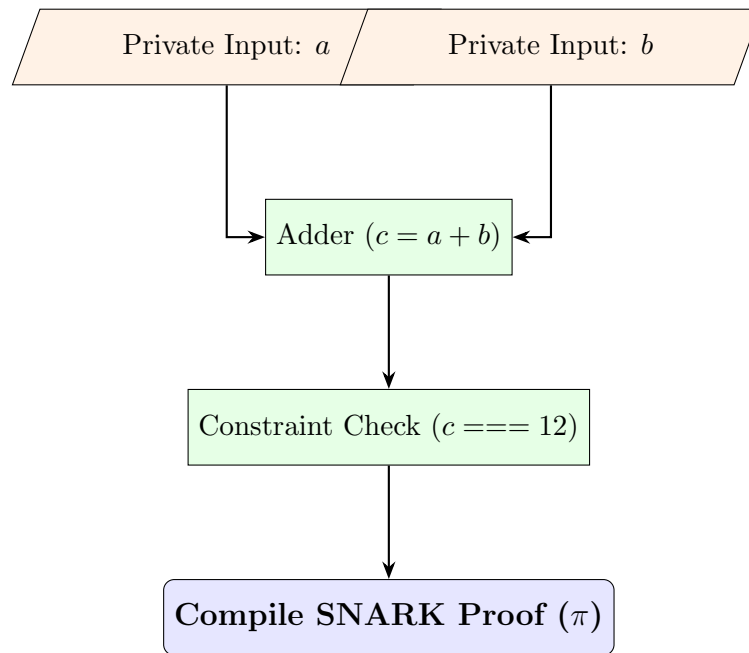
The Locked Box Analogy

Imagine a locked, impenetrable box with a slot on top and a lightbulb on the outside. Inside the box is a simple calculator programmed to only accept the number 12.

1. You walk up to the box privately and drop two pieces of paper inside (e.g., "5" and "7").
2. The calculator inside adds them together.
3. Because $5 + 7 = 12$, the box flashes a **Green Light** and spits out a cryptographic receipt.
4. A verifier sees the receipt. They know with absolute certainty you possess two numbers that equal 12. But they have absolutely no idea if your numbers were $6 + 6$, $10 + 2$, or $5 + 7$. Your inputs are safe inside the locked box.

Objective: Prove knowledge of two secret variables (a and b) that equal 12 without exposing them.

4.2 Data Flow Diagram



4.3 The Privacy Layer: Blinding Factors

One might assume an attacker could easily guess the values of a and b if the output is 12. However, Zero-Knowledge systems utilize **Blinding Factors**.

- During the execution of the proof, the Prover's software locally introduces randomly generated scalar values into the polynomials.
- This "blinds" the polynomial. The generated SNARK proof is completely distinct every single time it is run, even if the inputs (5 and 7) remain exactly the same.
- Because the proof is a 1.5 KB randomized hash bound by elliptic curve points, brute-forcing the input vector from the proof is computationally impossible.

4.4 Circom Source Code

```
1 pragma circom 2.0.0;
2
3 template SumIsTwelve() {
4     // 1. Declare signals (Data stays strictly local to the prover)
5     signal input a;
6     signal input b;
7     signal output c;
8
9     // 2. Circuit Logic: Calculate the sum locally
10    c <== a + b;
11
12    // 3. The Strict Constraint: Evaluates to True or Aborts
13    c === 12;
14 }
15
16 // Initialize the circuit
```

```
17 component main = SumIsTwelve();
```

Listing 1: Basic Arithmetic Constraint in Circom

4.5 Compilation and Execution: The Lifecycle of the Proof

When this code is compiled and executed via the terminal (using Circom and SnarkJS), the following lifecycle occurs:

1. **Compilation to WASM:** The ‘.circom’ file is compiled into WebAssembly (WASM). This allows the mathematical matrix (the witness) to be calculated seamlessly in a standard web browser or local Node.js environment.
2. **The Output Generation:** The user inputs ‘5’ and ‘7’. SnarkJS processes these against the Proving Key and outputs exactly two files:
 - **proof.json:** A 1.5 KB file containing the SNARK cryptographic payload.
 - **public.json:** A file containing *only* the public parameters (in this case, just the output 12).
3. **Data Traversal:** The ‘proof.json’ and ‘public.json’ files are sent over the network to the Verifier. The original inputs (‘5’ and ‘7’) remain permanently trapped in the user’s local RAM.
4. **Zero-Knowledge Verification:** The Verifier runs a mathematical check using their Verification Key against the two JSON files. If the cryptographic curves align, it returns ‘True’. At no point in the lifecycle did the Verifier’s system process, store, or view the actual numbers that produced the sum.

5 Implementation 2: E-KYC Age Verification (≥ 18)

5.1 In Plain English: What is actually happening?

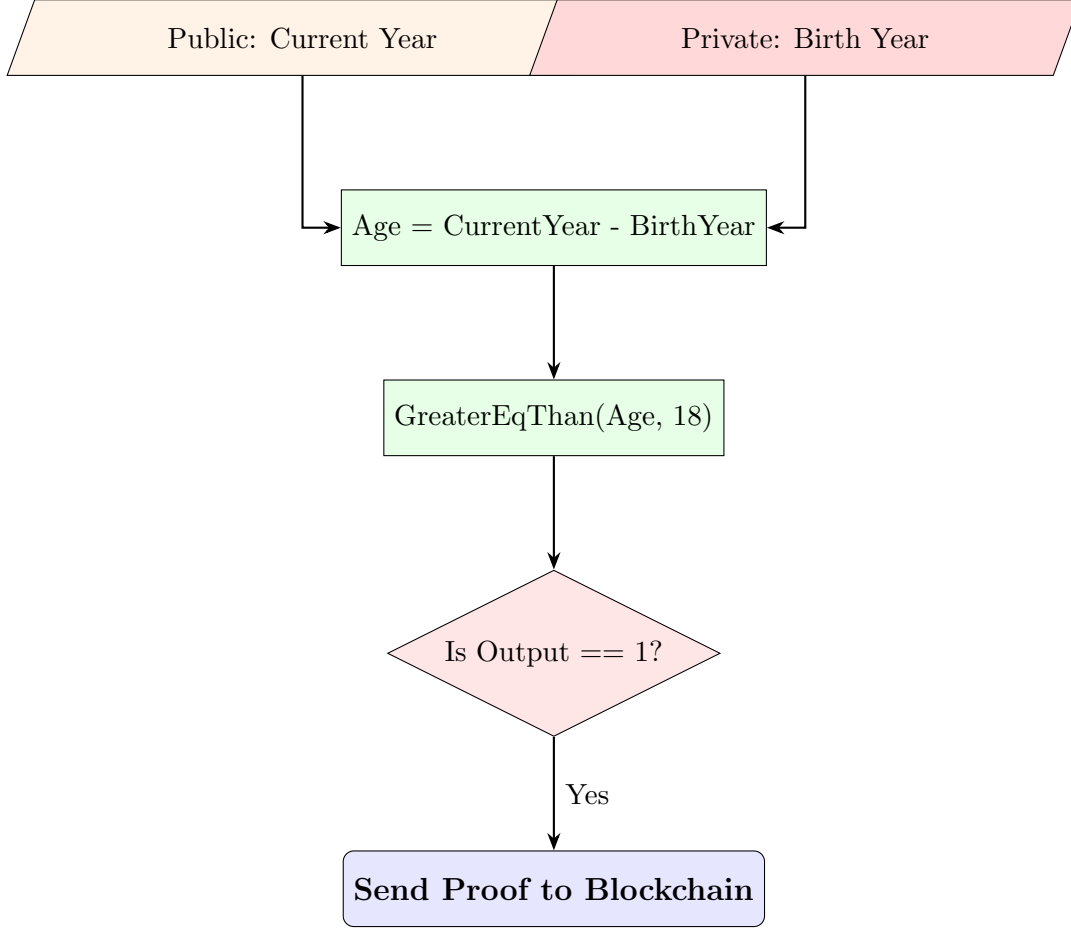
The Blindfolded Robot Analogy

Instead of a simple addition box, imagine you have a **Blindfolded Robot** sitting on your laptop.

1. You download a sealed, wax-stamped envelope from the Government (DigLocker) that contains your exact Birth Year.
2. You hand this sealed envelope to the Robot *locally* on your laptop. The envelope never goes over the internet.
3. The Robot breaks the seal, reads your birth year, subtracts it from the current year, and confirms you are over 18.
4. The Robot then **burns** the envelope and hands the bank a ”Proof Certificate.” The bank trusts the certificate because they know the Robot is mathematically programmed to only issue it if the government wax stamp was real, and the math was correct. The bank never sees the envelope.

Objective: Prove a user is legally allowed to access a restricted financial service without exposing their Date of Birth, utilizing government KYC infrastructure.

5.2 Verification Architecture



5.3 The Privacy Layer: Merkle Trees & ZK-Friendly Hashing

- **Why not SHA-256?** Standard hashing algorithms like SHA-256 are heavily optimized for CPUs, relying on bitwise operations. However, in a ZKP circuit, bitwise operations are incredibly expensive to represent as polynomials.
- **The Poseidon Hash:** To solve this, this implementation uses **Poseidon Hashing**, a ZK-friendly hash function natively designed over finite fields. It requires fewer than 300 constraints, making client-side proof generation exponentially faster.
- **DigiLocker Merkle Integration:** The user downloads a JSON credential from DigiLocker. The WASM client computes a Poseidon Merkle Root of the user's data (Name, Nationality, DOB) locally. The Circom circuit checks that the provided DOB accurately reconstructs the Merkle Root signed by the government, ensuring the data is authentic before executing the ≥ 18 constraint.

5.4 Circom Source Code


```

1 pragma circom 2.0.0;
2 include "circomlib/circuits/comparators.circom";
3
4 template AgeCheck(minAge) {
5     // Public parameter requested by the Smart Contract
6     signal input currentYear;
7
8     // Private parameter handled exclusively in local memory
9     signal input birthYear;
10    signal age;
11
12    // Logic: Compute Age
13    age <== currentYear - birthYear;
14
15    // Component: Initialize a 32-bit comparator from circomlib
16    component gte = GreaterEqThan(32);
17    gte.in[0] <== age;
18    gte.in[1] <== minAge;
19
20    // Constraint: The comparator out signal MUST be 1 (True)
21    gte.out == 1;
22 }
23
24 // Instantiate the main component, defining 'currentYear' as public
25 component main {public [currentYear]} = AgeCheck(18);

```

Listing 2: Age Verification using Circomlib Comparators

5.5 Compilation and Execution: The Lifecycle of the Proof

To transition this circuit into a functional e-KYC compliance tool, it undergoes a highly secure compilation and execution process:

1. **Local WebAssembly Compilation:** The backend compiles the age-check logic into WASM. When the user visits the financial institution’s web portal, this WASM file runs directly inside their browser’s local sandbox.
2. **The Output Generation:** The user locally links their DigiLocker credential containing their ‘birthYear’. The local WASM engine computes the age difference against the ‘currentYear’ (e.g., 2026) and generates the outputs:
 - **proof.json:** The cryptographic hash verifying the mathematics.
 - **public.json:** Contains the public `currentYear` and a boolean `true` indicating the ‘GreaterEqThan’ constraint passed. **Crucially, the exact birth year is completely absent from this file.**
3. **Data Traversal:** Only the two JSON files are transmitted to the financial institution’s servers or their deployed Ethereum Smart Contract. The DigiLocker document never leaves the user’s device.
4. **Zero-Knowledge Verification & Privacy Guarantee:** The financial institution (or Smart Contract) receives the ‘proof.json’. Because of the properties of the Groth16 zk-SNARK, the institution mathematically validates that the user is ≥ 18 .

The institution's centralized databases never store, process, or see the actual Date of Birth, achieving 100% compliance with strict data minimization laws like India's DPDP Act, effectively eliminating the risk of PII breaches.